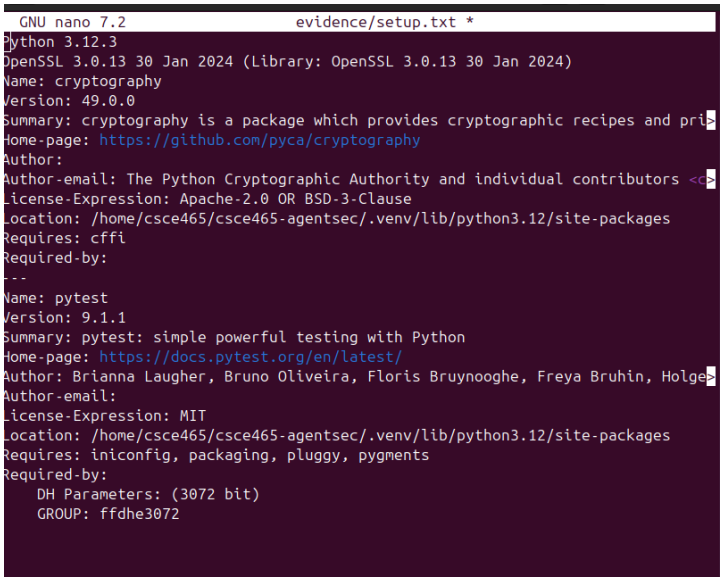
To set up this lab I had to create a virtual environment and download the corre>
The gateway and node were simulated in one process, so network sockets werent n>

Python version-3.12.3
OpenSSL-3.0.13
cryptography/pytest 49.0.0/9.1.1
DH parameters 3072 bit; GROUP: ffdhe3072



Task 1 - Encryption alone
The baseline encrypts the fixed JSON command with AES-CTR and no MAC. A relay receives the ciphertext without the key. Knowing the fixed command layout, it changes the four bytes corresponding to READ into LIST. It then sends the modified cipher text twice. The receiver only decrypts and parses the JSON; it has no integrity or replay check.
Original action bytes: 52454144
Replacement bytes: 4c495354
XOR difference 1e0C1210
Receiver process: {action: READ, path: notes.txt}
Receiver process: {action: LIST, path: notes.txt}
Receiver process: {action: LIST, path: notes.txt}

CTR produces C = P XOR S, where S is the keystream. At the targeted positions the relay applies C' = C XOR READ XOR LIST. Decryption then gives LIST because the original READ bytes cancel. The XOR difference is 1e0c1210. Neither the modified command or its replay is detected. The root cause is missing authentication and missing receiver sequence state, not an attacker recovering the AES key.

Task - Authenticated Diffie-Hellman
Design and trust assumptions

Each party has 3072-bit RSA signing key. The simulation directly supplied each party with the expected peers RSA public key, establishing trust independently of received handshake messages. RSA keys are generated once per demonstration process and reused across its sessions. They sign the exchange rather than encrypt application messages. Each session uses a fresh finite field DH private value and a fresh 16-byte nonce.

Canonical transcript and signature
The transcript has eight fields in this order: protocol label CSCE465-HS-v2, group identifier ffdhe3072, gateway identity, node identity, gateway DH public value, node DH public value, gateway nonce, and node nonce. Each field begins with a four byte big endian length. Each public value is encoded in 384 big endian bytes. The parser rejects incomplete fields, trailing bytes, wrong fixed values, unexpected identities and incorrect public value or nonce widths before hashing.

Both parties hash the same transcript to obtain TH. Each uses RSA-PSS with SHA-256 to sign role || TH. Verification uses the expected peer role and trusted peer RSA key. Each party also checks that the transcript contains its own current public value and nonce. This binds acceptance to the current local session.

Security interpretation
Signing both identities, both DH values, and both nonces prevents basic exchange substitution when the trusted RSA keys are correct. Distinct role prefixes prevent accepting one roles signature as the others. Fresh local DH values and nonces make an old transcript fail current session checks. Without the signatures, an active attacker could substitute public values and establish a separate shared secret with each endpoint. Ephemeral DH supports forward secrecy against later signing key compromise, provided old ephemeral private values are discarded. Different KDF labels separate directions and encryption/MAC purposes.

Task 3 encrypt then mac records

Record Encoding
Version-1 byte version 1
Direction 1 byte; 0=gateway to node, 1=node to gateway
Sequence 8 bytes; big-endian unsigned integer
Message type 1 byte; demo uses type 1
Ciphertext_length 4 bytes; big endian unsigned integer
IV- 8-byte session ID followed by 8-byte sequence
Ciphertext same length as plain text
Tag 32 byte HMAC-SHA-256 result

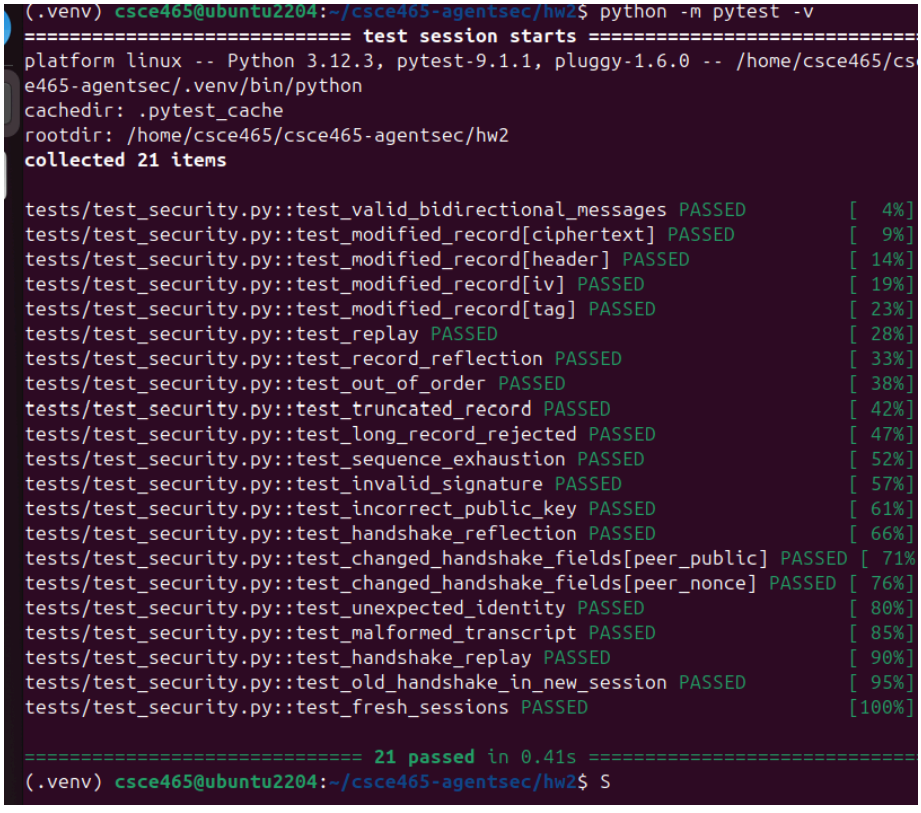
Sender and receiver behavior
Each direction starts at sequence 0 and has separate encryption and MAC keys. seal() constructs the record and advances the senders sequence. The receiver first validates record

framing, then verifies the complete MAC with the library. It checks the authenticated version, direction, exact next sequence, and expected IV before decrypting. Only a successfully accepted record advances the receiver state and returns plaintext. An error raises RecordError without returning plaintext

Observed communication and replay rejection
Gateway-> node: (1,b'Read notes.txt')
Node-> Gateway: (1:b'Ok')
Next gateway record(1, b'List')
Replay rejected: unexpected sequence

Counter allocation limitation
This implementation limits plaintext to 16 bytes per record. AES-CTR increments the counter for each AES block. With the required IV=session_id || sequence and sequence increments of one, a two-block record starting at sequence zero would consume the counter that the next record starts with. Unique starting IVs alone would therefore not prevent repeated keystream blocks. The one block limit prevents this overlap while retaining the prescribed record format.

Task 4- Adversarial tests
Execution evidence



This is the result shared from the VM. The table groups the 21 trest cases including parameterized cases. Each negative test checks a specific exception message; several also verify that failed input did not advance state.

What these tests establish

The modification tests show that changed authenticated bytes are rejected the replay and ordering tests exercise exact next sequence enforcement. Reflection tests show the effect of directional keys and expected role signatures. Handshake tests exercise trusted key verification field binding identity validation and transcript parsing. These results support the specified tested behaviors; they do not prove that every attack or implementation bug is excluded.

Security note

The attacker in this demonstration can observe, inject, reorder, reflect, and replay messages exchanged between the simulated gateway and node. The attacker does not initially know their private signing keys or session keys. Peer RSA public keys are already trusted through the simulation's local configuration, rather than learned from an untrusted handshake. The goal is confidentiality, record integrity, authenticated key establishment and ordered replay-resistant delivery. Endpoint compromise, denial of service, traffic analysis, and deciding whether a requested tool action is safe are outside these guarantees. All commands are harmless simulated messages rather than tool executions.

CTR encrypts plaintext by XORing it with a keystream generated from the AES counter blocks. Reusing a counter block under the same key repeats the keystream. XORING the affected ciphertext portions then reveals the XOR of the corresponding plaintext portions; known plaintext can expose the other portion. The assignment places the sequence number in the low eight bytes of the starting counter. A multi-block record can therefore consume counters assigned to later records. This implementation limits protected plaintext to one AES block, or sixteen bytes, to avoid the overlap while following the specified IV format. It rejects sequence exhaustion and requires session objects to retain their counters.

The receiver validates framing and verifies the HMAC before decrypting. The MAC covers the header, IV, and ciphertext, so an attacker cannot alter those values without detection under the assumed security of HMAC. The library performs secure MAC comparison. Invalid records raise exceptions and return no plaintext. An authenticated record must also match the expected version, direction, sequence, and IV. Receiver state advances only after successful acceptance. Replay resistance comes from that state, not from the MAC alone, an old unmodified record still has a valid tag but is rejected because its sequence is no longer expected.

Encryption and authentication use independent derived keys. Separate derivation labels also distinguish gateway-to-node traffic from node-to-gateway traffic. This keeps the purposes and directions distinct rather than depending on one shared key for everything. The handshake authenticates fresh DH public values and nonces with RSA-PSS signatures over role-prefixed

transcript hashes. Both identities are included, and expected roles prevent basic reflection. Length prefixes prevent ambiguous transcript field boundaries. Without signatures, an active attacker could substitute DH public values and create separate sessions with the two parties. Fresh ephemeral DH also provides forward secrecy against later signing-key compromise if ephemeral private values are discarded.

AES-GCM is an authenticated encryption mode that combines encryption and integrity protection in one construction. The record header can be supplied as associated data, protecting it without encrypting it. Compared with the separate CTR and HMAC layer here, a maintained AEAD implementation reduces the amount of composition code the application must get right. It still needs correct nonce management, trusted session establishment, and error handling.

A valid protected record establishes that its authenticated bytes came from a holder of the appropriate session keys; it does not establish that the requested action is sensible or allowed. An authorized or compromised endpoint could request deletion of an important file, disclosure of confidential data, or another harmful operation. The application must independently enforce permissions, validate arguments, restrict accessible resources, and apply least privilege. This demonstration only prints mock commands. Its passing adversarial tests support the tested protocol behaviors, while the one-block limit, trusted local key configuration, and sequential in-memory state remain explicit implementation limitations